CRIMEVISIONAI

Technology Stack

Folder 2 | Verified implementation baseline | Prepared 23 October 2026

Project: CrimeVisionAI — Multimodal Forensic Evidence Analysis and Investigation Support System

Document status: Project-specific draft based on the implementation information currently available. Verify placeholders and run-dependent results before formal submission.

1. Technology inventory

Layer / Area	Technology / Component	Role / Notes
Frontend	HTML, CSS, JavaScript, Jinja2 templates	Browser-based forensic dashboard; no React/build pipeline reported.
Web backend	Python, Flask 3.x app factory	Routes, request validation, case/evidence workflow and API orchestration.
Database	SQLite via raw sqlite3	Local persistence; tables reported: cases, evidence, analysis_results.
Speech transcription	Whisper base (Hugging Face)	Audio transcription with timestamps; processed in 30-second chunks.
Object detection	RT-DETR (rtdetr-l.pt)	Image/video object detection; configurable confidence presets.
Image-text similarity	CLIP (openai/clip-vit-base-patch32)	Ranks scene/category labels; output is a model estimate, not proof.
Image captioning	BLIP	Generates image/video-frame captions.
LLM integration	Groq API, openai/gpt-oss-20b	Assists with structured interpretation, event/timeline generation and investigator Q&A.
File storage	Local filesystem	Evidence uploads and case-scoped upload paths; upload limit reported as 100 MB.
Runtime	Local CPU with automatic CUDA detection	Hardware availability affects latency; production deployment configuration not yet established.

2. Application and API surface

Area	Reported endpoints / behavior	Notes
Audio	/api/audio/analyze	Whisper transcription; timestamped output and LLM-assisted event structuring.
Image	/api/image/analyze	RT-DETR detections, CLIP scene labels, BLIP caption, cautious interpretation.
Video	/api/video/analyze	Frame sampling at 1 FPS, up to 180 seconds / 180 frames.
Cases	Case CRUD-style API endpoints	Exact route names should be confirmed from the repository.
Evidence	Upload, analyze and media endpoints	Case-scoped evidence storage and media retrieval.
Investigator Q&A	Case chat endpoint	Known grounding issue; answer completeness needs regression tests.

3. Configuration and runtime

Known configuration: Flask 3.x app factory; local development entry point run.py; reported bind 127.0.0.1:5000 with debug=True; MAX_CONTENT_LENGTH 100 MB; local model execution with automatic CUDA detection; Groq integration configured separately. Debug mode is appropriate only for trusted local development and should be disabled for any shared/production deployment.

4. Dependencies and reproducibility

Use the repository's actual requirements.txt and environment configuration as the source of truth. The original template ZIP contains documentation only, not the CrimeVisionAI source repository, so dependency versions, environment variable names, exact route signatures and model download behavior must be checked against the code before final release. Never include real API keys in the repository.

5. Architecture decision rationale

Flask and a server-rendered vanilla frontend reduce build complexity for a student prototype. SQLite and local storage simplify local demonstrations. Local vision/speech models provide modality-specific processing while Groq handles selected language tasks. These choices are suitable for a prototype but require queueing, access control, storage abstraction, observability and deployment hardening for larger-scale use.